

# CS698B: Fundamentals of Data Engineering (Part I)

## 2026-27-I, IIT Kanpur – Problem Sheet 5



### General Instructions:

1. This problem sheet need not be submitted.
2. The questions are meant for practice and self-assessment.
3. Please use this resource honestly to assess your understanding of concepts.

**Q1 ( Be the Truth Machine )** Write the output of the following code that explores truthy Python values.

**Recall:** the backslash indicates *line-continuation* and is used to break long lines into readable chunks.

```
faculty = [ ( "dee", "" ), ( "boo", "cs" ), ( "bro", "ee" ) ]
affiliated = [ ( i, name ) for i, ( name, dept ) \
                in enumerate( faculty ) if dept ]

print( affiliated )

flags = [ 0, "", [], 3 ]
print( [ x for x in flags if x ] )
```

**Q2 ( Be Pythonic )** Rewrite the code (except for the first line) in a **single line of idiomatic Python**.

```
word_list = [ "deebo", "wishes", "you", "all", "the", "best" ]
out = ""
i = 0
while i < len( word_list ):
    if i > 0:
        out = out + " "
    out = out + word_list[ i ]
    i = i + 1
print( out )
```

**Q3 ( Be Pythonic )** Rewrite the code (except for the first line) in a **single line of idiomatic Python**.

**Hint:** you may find the unpacking operator `*` and customizable arguments in `print()` to be helpful.

```
num_list = [ 10, 20, 30 ]
i = 0
while i < len( num_list ):
    print( i, num_list[i] )
    i = i + 1
```

**Q4 ( Be Pythonic )** Rewrite the code (except for the first line) in a **single line of idiomatic Python**.

```
a, b = [ 1, 2 ]
tmp = a
a = b
b = tmp
```

## CS698B: Fundamentals of Data Engineering (Part I)

### 2026-27-I, IIT Kanpur – Problem Sheet 5

**Q5 ( Be the Pythonic Detective )** Consider the following pieces of code and answer the questions. **Note:** the Python builtin function `any()` operates over iterables (lists, tuples etc) and returns `True` if at least one value in that iterable is `True` or a truthy value. The function will return `False` on iterables that are empty or that contain only falsy values.

| SNo | Code                                                                                                                                                            |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A   | <pre>def has_digit( string ):     for char in s:         if char in "0123456789":             return True     return False</pre>                                |
| B   | <pre>def has_digit( string ):     return any( char.isdigit() for char in s )</pre>                                                                              |
| C   | <pre>def has_digit( string ):     for i in range( len( string ) ):         if s[i] &gt;= '0' and s[i] &lt;= '9':             return True     return False</pre> |

1. Will the three functions always give the same output on ASCII strings?
2. Which function is written in the most Pythonic manner? Why?
3. The `str` class has a method called `isdigit()`. How does it differ from the `has_digit()` function? Give a string on which both functions give different outputs.

**Q6 ( Be the Pythonic Detective )** Consider the following pieces of code. They are supposed to take a list of `( name, score )` pairs and create a dict mapping each name to its max score. Which one is buggy? Design a revealing test case. Which one is most Pythonic? Why? Which one is the least Pythonic? Why? Note that the `get()` method for dictionaries takes in a default value to return in case a key does not exist.

| SNo | Code                                                                                                                                                                                                                                                                                                                                           |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A   | <pre>def best( scores ):     max_dict = {}     for name, score in scores:         if name not in max_dict or score &gt; max_dict[ name ]:             max_dict[ name ] = score     return max_dict</pre>                                                                                                                                       |
| B   | <pre>def best( scores ):     max_dict = {}     for name, score in scores:         max_dict[ name ] = max( score, max_dict.get( name, score ) )     return max_dict</pre>                                                                                                                                                                       |
| C   | <pre>def best( scores ):     return { name: score for name, score in scores }</pre>                                                                                                                                                                                                                                                            |
| D   | <pre>def best( scores ):     max_dict = {}     for i in range( len( scores ) ):         name = scores[i][0]         score = scores[i][1]         if name in max_dict.keys():             if score &gt; max_dict[ name ]:                 max_dict[ name ] = score         else:             max_dict[ name ] = score     return max_dict</pre> |

## CS698B: Fundamentals of Data Engineering (Part I)

### 2026-27-I, IIT Kanpur – Problem Sheet 5

**Q7 ( DePythonify )** The code below is meant to count how many times does a word appear in a list of strings. The code is correct and Pythonic too but its intent is a bit obscured due to use of the sophisticated EAFP (*easier to ask for forgiveness than permission*) style of programming for a simple task that doesn't need it. Rewrite the code to be Pythonic but also clear. You may find the default argument in the `get()`

```
def counts( word_list ):
    word_dict = {}
    for word in word_list:
        try:
            word_dict[ word ] += 1
        except KeyError:
            word_dict[ word ] = 1
    return word_dict
```

method for dictionaries helpful in this rewrite. **You should be able to rewrite the entire try-except block in a single line of Pythonic code.** In case you are curious, the opposite of EAFP is the more conservative style of programming is called LBYL (*look before you leap*). EAFP optimistically assumes nice conditions and handles non-nice situations using exception handling. LBYL ensures niceness before proceeding.

**Q8 ( Reorder the Code )** The code below was meant to take a list of item-quantity pairs and print the items in alphabetical order along with the total quantity of that item ( items may repeat ).

```
pairs = [ ( "bread", 2 ), ( "apple", 1 ), ( "bread", 3 ), ( "apple", 4 ) ]
```

As happens every week, the code got jumbled up and needless lines may have gotten added due to Deebea's eagerness to help Deeborah. Reorder the lines in correct order ( you may have to use a line more than once ), indent properly, identify the unnecessary line(s), and print the output for the above input.

```
for i in range( len( pairs ) ):
for item, qty in pairs:
item, qty = pairs[ i ]
parts = [ f"{item}:{totals[ item ]}" for item in sorted( totals ) ]
print( ", ".join( parts ) )
print( parts )
totals = {}
totals[ item ] = totals.get( item, 0 ) + qty
totals[ item ] = totals[ item ] + qty
```